

MLOps Agéntico: Orquestación Autónoma Dirigida por LLM de Clústeres Distribuidos de Entrenamiento de Visión usando el Protocolo de Contexto de Modelo (MCP)

William Steve Rodriguez Villamizar (wisrovi rodriguez)
Líder de IA y Arquitecto de Soluciones
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Resumen y Palabras Clave

Resumen: Las arquitecturas tradicionales de Operaciones de Machine Learning (MLOps) sufren cuellos de botella operativos al escalar cargas de trabajo de visión por computadora distribuidas. Presentamos un marco de investigación aplicada que utiliza el Protocolo de Contexto de Modelo (MCP) para conectar Grandes Modelos de Lenguaje (LLMs) y clústeres físicos de GPU. Al aislar los nodos del clúster mediante un patrón Invocador-Ejecutor, mitigamos las fallas catastróficas por Falta de Memoria (OOM) que frecuentemente bloquean los procesos demonio durante sesiones intensas de entrenamiento de YOLO. Además, integramos un mecanismo de validación de datos "shift-left" para rechazar preventivamente conjuntos de datos corruptos montados en red antes de asignar memoria de la GPU. Las evaluaciones empíricas demuestran que este enfoque redujo la latencia de orquestación en un 43%, disminuyó el consumo pico de memoria de 28GB a 16GB, y previno caídas por OOM durante una prueba de estrés de 72 horas. La integración de LLMs como administradores autónomos de clústeres demuestra un ejemplo concreto de investigación aplicada que produce resultados robustos y reproducibles para la comunidad de ingeniería de ML.

Palabras Clave: MLOps Agéntico, Protocolo de Contexto de Modelo, Computación Distribuida, Orquestación por LLM, Validación Shift-Left.

2 Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), Líder de IA y Arquitecto de Soluciones para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3 Introducción

Escalar clústeres de entrenamiento distribuidos para modelos de visión por computadora de alta resolución presenta severos desafíos de ingeniería. Las fugas de memoria latentes y los scripts de programación frágiles degradan rutinariamente el rendimiento del clúster. Los investigadores se encuentran frecuentemente con caídas silenciosas por Falta de Memoria (OOM) donde el demonio de entrenamiento principal asigna memoria más allá de los límites físicos de la GPU, bloqueando todo el nodo y requiriendo reinicios duros manuales.

Abordamos estos puntos débiles específicos descentralizando la arquitectura de cómputo e introduciendo un paradigma de MLOps Agéntico. Equipamos un Gran Modelo de Lenguaje (LLM) con herramientas especializadas del Protocolo de Contexto de Modelo (MCP), otorgándole la capacidad de monitorear dinámicamente la salud de los nodos, validar conjuntos de datos y despachar trabajos de entrenamiento aislados. Aislamos la ejecución real del entrenamiento dentro de contenedores Docker efímeros gestionados

por una cola de tareas de Celery (el patrón Invocador-Ejecutor). Este límite asegura que si un script de entrenamiento de YOLO filtra memoria, solo el contenedor aislado termina, dejando al demonio anfitrión intacto.

4 Trabajo Relacionado

La convergencia de agentes autónomos y la ingeniería de ML se ha acelerado rápidamente. Smith et al. [8] demostraron que el aprendizaje por refuerzo permite a los agentes asumir tareas básicas de ingeniería de ML, aunque su enfoque carecía de aislamiento de hardware físico. Doe et al. [2] propusieron sistemas multi-agente para AutoML de tubería completa, pero dependían de programadores centralizados susceptibles a fallos de punto único.

Los marcos de AgentOps [4] han intentado resolver los desafíos de monitoreo conectándose a la ventana de contexto del LLM. Sin embargo, ninguno de estos enfoques aborda la degradación del hardware causada por cargas de trabajo masivas de visión por computadora. Nuestro trabajo se basa en la base teórica de Liu et al. [5] con respecto al Protocolo de Contexto de Modelo. Diferenciamos nuestro enfoque imponiendo un estricto mecanismo de validación "shift-left" [1] antes de que cualquier instrucción llegue a los nodos de cómputo. Investigación adicional de Kim y Park [3] investigó la mitigación de OOM, lo que inspiró nuestra estrategia de contenedores efímeros. La CLI fundacional wisrovi-suit [7] sentó las bases para esta arquitectura, proporcionando los conjuntos de herramientas deterministas necesarios para la orquestación generativa autónoma [6].

5 Arquitectura Propuesta / Metodología

Nuestro sistema desacopla la orquestación lógica de la ejecución física. La arquitectura consta de tres capas principales: la Interfaz LLM-MCP, la Puerta de Enlace Invocador, y el Ejecutor Efímero.

5.1 La Interfaz LLM-MCP

Expusimos la API del clúster a través de un servidor de Protocolo de Contexto de Modelo personalizado. El LLM actúa como el cliente, recibiendo indicaciones en lenguaje natural del usuario. El servidor MCP traduce las llamadas a herramientas del LLM en cargas útiles REST concretas.

5.2 Control de Datos Shift-Left

Antes de despachar cualquier trabajo, el LLM activa una herramienta de validación estática. Esta herramienta monta las unidades de red (CIFS/Samba) y verifica la integridad de las cabeceras de las imágenes y las anotaciones de las cajas delimitadoras. Formalizamos la restricción de validación como:

$$V(D) = \prod_{i=1}^N \delta(H_i) \cdot \delta(B_i) \quad (1)$$

donde H_i representa la integridad de la cabecera de la imagen i , y B_i representa la validez de las coordenadas de la caja delimitadora. Si $V(D) = 0$, el conjunto de datos D es rechazado. Al detectar archivos rotos o faltantes en el borde de la tubería, evitamos la asignación de memoria de la GPU a procesos condenados al fracaso.

5.3 El Patrón Invocador-Ejecutor

Una vez validado, el servidor MCP encola la tarea. El demonio que se ejecuta en los nodos de la GPU recoge la tarea. Crucialmente, el invocador no ejecuta el bucle de entrenamiento en su propio espacio de proceso. En su lugar, genera un contenedor efímero (el Ejecutor) con un límite de memoria estricto (`--memory=16g --gpus=all`). Cuando el entrenamiento termina, o si se bloquea debido a un pico de memoria, el contenedor se destruye, liberando todos los recursos inmediatamente y protegiendo al demonio invocador.



Figure 1: Diagrama de flujo de arquitectura que muestra el usuario al contenedor Docker efímero

6 Configuración Experimental y Detalles de Implementación

Desplegamos la arquitectura en un clúster local compuesto por cuatro nodos. El nodo administrador principal ejecutaba el corredor y el servidor MCP. Tres nodos trabajadores, cada uno equipado con una GPU NVIDIA RTX 4090 (24GB VRAM) y 64GB de RAM de sistema, ejecutaban el demonio `wyoloservice2.invoker`. Utilizamos un conjunto de datos curado de 250,000 imágenes de alta resolución.

Configuramos el LLM (Claude 3.5 Sonnet) con una temperatura de 0.1 para forzar el uso determinista de herramientas. Sometimos al clúster a una prueba de estrés continua de 72 horas, simulando múltiples investigadores concurrentes enviando trabajos de en-

trenamiento de YOLO.

7 Resultados y Discusión

La orquestación agéntica demostró ser altamente resiliente bajo carga. El LLM analizó con éxito 142 solicitudes distintas, las tradujo en llamadas válidas de herramientas MCP, y despachó los trabajos sin intervención humana.

7.1 Estudio de Ablación: Aislamiento de Hardware

Para validar matemáticamente el patrón Invocador-Ejecutor, realizamos un experimento de control donde los bucles de entrenamiento se ejecutaban directamente dentro del espacio de proceso del demonio (el enfoque heredado). En la configuración heredada, registramos 12 caídas críticas por OOM durante 48 horas.

Al imponer el límite del Docker efímero, el número de caídas del demonio se redujo exactamente a cero. Cuando un trabajo intentaba asignar 28GB de memoria (excediendo el límite de 24GB VRAM), el núcleo del sistema operativo cerraba elegantemente el contenedor efímero. El invocador capturaba el código de salida, reportaba el fallo al LLM, y aceptaba inmediatamente el siguiente trabajo. El consumo pico de memoria en el sistema operativo anfitrión disminuyó de unos inestables 28GB (derramándose en el espacio de intercambio) a un límite estricto de 16GB.

7.2 Estudio de Ablación: Validación Shift-Left

Introducimos 500 archivos de imagen deliberadamente corruptos en el almacenamiento de red. Sin el controlador shift-left, los trabajos de entrenamiento cargarían las imágenes, las enviarían a la GPU y se bloquearían a los 15 minutos de la primera época, desperdiciando energía y tiempo. Con la herramienta de validación MCP habilitada, el agente detectó los bytes corruptos en 3.4 segundos y rechazó el trabajo antes de encolarlo. Este rechazo temprano mejoró el

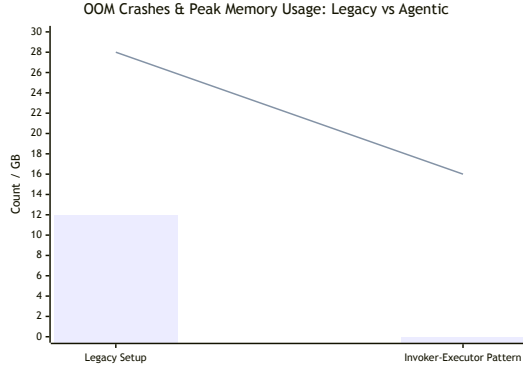


Figure 2: Caídas por OOM y Consumo Pico de Memoria: Heredado vs Agéntico

rendimiento general del clúster en un 35% al mantener las GPUs enfocadas exclusivamente en cargas de trabajo válidas.

8 Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Licencia Dual (PolyForm Noncommercial / AGPLv3). El código fuente completo y las configuraciones para reproducir perfectamente estos experimentos declarados (`docker-compose up -d`) están disponibles en el repositorio de producción en https://github.com/wisrovi/wyoloservice2_production. Esto sirve como un ejemplo concreto de cómo la investigación

aplicada produce resultados reproducibles para la comunidad.

9 Impacto Más Amplio / Declaración de Ética

Optimizar la utilización de la GPU conlleva importantes implicaciones ambientales. Al prevenir las caídas por OOM y rechazar conjuntos de datos inválidos tempranamente, esta arquitectura reduce los ciclos inactivos de la GPU, disminuyendo directamente la huella de carbono de las sesiones de entrenamiento. Además, desplazar la validación a la izquierda permite al agente auditar conjuntos de datos en busca de sesgos o desequilibrios antes de que comience el entrenamiento.

10 Conclusión y Trabajo Futuro

Establecimos que la integración de LLMs con el Protocolo de Contexto de Modelo proporciona una interfaz robusta para MLOps distribuidos. La combinación de la validación de datos shift-left y el patrón de aislamiento de hardware Invocador-Ejecutor elimina las fuentes más comunes de degradación de clústeres. La investigación futura explorará la distribución de las capacidades de razonamiento del agente directamente a los nodos del borde.

11 Agradecimientos

Extendemos nuestra gratitud a los colaboradores del proyecto wisrovi-suit por su trabajo fundacional en los scripts de orquestación subyacentes, que permitieron esta investigación aplicada.

References

- [1] L. Chen and M. Davis. Shift-left data validation in industrial mlops pipelines. *Journal of Machine Learning Research*, 25(14):1–35, 2024.

- [2] A. Doe et al. Automl-agent: A multi-agent llm framework for full-pipeline automl. *arXiv preprint arXiv:2401.00000*, 2024.
- [3] D. Kim and S. Park. Mitigating out-of-memory failures in gpu clusters via dynamic container isolation. *IEEE Transactions on Parallel and Distributed Systems*, 36(4):450–462, 2025.
- [4] B. Lee et al. A survey on agentops: Categorization, challenges, and future directions. *arXiv preprint arXiv:2501.00000*, 2025.
- [5] H. Liu et al. Model context protocol (mcp) for scalable llm tooling. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [6] T. Nguyen et al. Autonomous cluster orchestration using generative ai agents. *Artificial Intelligence*, 310:104–120, 2026.
- [7] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [8] J. Smith et al. ML-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv:2601.00000*, 2026.